

bigspy 运行速度 / 性能测试报告

- 日期：2026-09-21
- 被测版本：合并后分支 `dev @ 6154fd6` (Merge branch 'master' into dev)
- 相对上一版（332a043）吸收了 `master` 的 40 个提交，其中最影响性能的是 **JAX 1D 卷积核**（取代稠密 3129^2 矩阵）与 `likelihood` 的 NaN/Inf 防护
- 运行方式：conda 环境 `bigspy` (Python 3.11.16)
- 原始日志：`out/example_run.log`、`out/benchmarks/*.log`（含 `ALL.log` 汇总、`pytest.log`）

1. 测试环境

项目	值
CPU	AMD Ryzen 5 3550H, 4 核 / 8 线程, 最高 3.7 GHz
内存	14 GiB
磁盘	NVMe SSD
系统	Fedora Linux 44, 内核 7.2.5
Python	3.11.16 (conda env bigspy)
numpy / scipy	2.4.6 / 1.17.1
astropy / lmfit	8.0.1 / 1.3.4
ultranest	4.5.2 (HDF5 后端, h5py)
jax / jaxlib	0.10.2 / 0.10.2 (CPU 后端)

仓库自带的 `benchmarks/benchmark_report.md` 基线是 80 核 Xeon Gold 6248 + 754 GB 内存，本机为 4 核/8 线程移动 CPU，绝对值不可直接对比；本报告全部为本机实测。

2. 测试方法

1. 单元测试： `python -m pytest`（合并后代码）。
2. 端到端： `/usr/bin/time -v` 运行 `example/run_bigspy_jax.py`（SpecFit + 三次 UltraNest）；分段耗时另用各 `chain` 目录的 `debug.log` 时间戳交叉核对。
3. 微基准：从仓库根依次、串行运行 `benchmarks/benchmark_*.py` 全部 8 个脚本，记录墙钟与退出码。

3. 单元测试

161 passed in 52.14s

(含合并时新增的 `tests/test_sampler.py::TestRun::test_each_run_writes_its_own_debug_log`，以及适配新 JAX API 的 `tests/test_jax.py::TestBuildConvKernel`。)

4. 端到端全流程（`example/run_bigspy_jax.py`）

4.1 总览

指标	合并后（本次）	合并前（332a043）
总墙钟	607 s（10 分 07 秒）	547 s（9 分 07 秒）
用户 CPU	1795.1 s	2173.1 s
系统 CPU	71.2 s	35.7 s
平均 CPU 利用率	307%	403%
峰值内存 (Max RSS)	3.92 GiB	3.16 GiB
Swap	0	0

4.2 分阶段耗时

阶段	起止	墙钟	似然评估次数	吞吐
SpecFit (mode2)	—	0.1 s	—	—
主 MCMC — DelayedExp, <code>n_live=400</code>	11:12:31→11:13:45	74 s	24,112	~326 /s
自定义 SFH — DelayTau, <code>100→200</code> ，max 3000	11:13:47→11:13:59	12 s	3,025	~252 /s
DPL, <code>n_live=199→398</code>	11:13:59→11:22:30	511 s（8.5 min）	1,314,827	~2,573 /s
绘图 / 保存	—	~10 s	—	—

DPL 段仍占绝对主导（约 84% 墙钟）。合并后它的似然调用量从 59.3 万涨到 **131 万**（UltraNest 自动加宽 199→398），因此总时间反而比合并前更长。**这不是 JAX 变慢，而是该次采样的路径变化**（见 § 6）。

4.3 实际科学输出

SpecFit (mode2)（与合并前完全一致）：

`v_e = 146.7 ± 5.2 km/s   v_d = 67.7 ± 7.6 km/s   E(B-V) = 0.1880 ± 0.0155`
`p1 = 0.5929,   p2 = -0.0650`

主 MCMC (DelayedExp, `logZ = -1638.40`，posterior N=4710，ESS=932)：

`t0        =   1.0912   [+1.4137 / -0.7192]   Gyr`
`tau       =   3.6138   [+0.2822 / -0.5352]   Gyr`
`logZsun = -0.9223   [+0.0441 / -0.0410]`
`Z         =   0.00237   (Z_solar = 0.02)`

模型对比：

模型	参数数	log Z	$\Delta\log Z$
DelayedExp	3	-1638.40	+0.0
DelayTau	2	-1636.79	+1.6
DPL	4	-1636.29	+2.1

结论与合并前一致：略偏好 DelayTau/DPL，但 $\Delta\log Z \approx 2$ ，证据很弱。

5. Benchmark 套件（全部 8 个，合并后）

脚本	主题	墙钟	本机加速比
benchmark_sfh	P4 SFH evaluate_batch	3 s	4.0–6.7×
benchmark_csp	P3 CSP build_batch	39 s	合成 1.9–2.3× / 真实 1.5–16.5×
benchmark_conv_matrix	P1 卷积矩阵构建 + 缓存	10 s	构建 2.4–22.3× / 批量 1.4–4.3×
benchmark_likelihood_interp	P2 插值/掩码预计算	3 s	插值 1.3–2.8× / call_batch 1.5–2.1×
benchmark_mean_filter	P5 滑动窗口向量化	2 s	4.6–48.5×
benchmark_specfit_hotpath	P6 SpecFit 热路径	16 s	calz 2.6× / mode2 10.3× / 整体 1.74× / 真实谱 4.11×
benchmark_mcmc_pipeline	组合 P1+P2+P3+P4	220 s	2.1–2.5×
benchmark_jax	JAX JIT vs NumPy	14 s	3.1–4.0×
合计		~307 s	8/8 退出码 0

5.1 benchmark_jax（合并带来的最大变化）

N	NumPy (s)	JAX 1st (s)	JAX mean (s)	Speedup	Δ_{rel}
10	0.0469	0.5810	0.0141	3.3x	1.59e-04
50	0.0662	0.5570	0.0211	3.1x	4.90e-04
200	0.1534	0.6360	0.0385	4.0x	3.78e-04
500	0.3530	0.5899	0.0964	3.7x	4.66e-04
1000	0.6534	0.6650	0.1814	3.6x	4.80e-04

对比合并前：JAX 稳态耗时全面下降（N=1000：0.286 s $\rightarrow$ 0.181 s），加速比从 2.3–3.3× 提升到 **3.1–4.0×**。这正是 master 用 **1D 卷积核（约 30 元素）** 替代稠密 **3129² 矩阵** 的效果。

5.2 benchmark_mcmc_pipeline（真实 MaNGA + 真实 SSP）

N	old (s)	new (s)	speedup	max dchi2	rel
10	0.7815	0.3805	2.1x	3.6e-11	9.5e-16
50	1.3175	0.5874	2.2x	1.5e-10	1.5e-15
200	3.8353	1.5545	2.5x	1.2e-10	3.3e-15
400	7.1639	3.0733	2.3x	2.6e-10	3.0e-15
1000	17.4348	7.4596	2.3x	1.7e-10	6.6e-15

5.3 其余脚本

```
benchmark_sfh:          4.0-6.7x,  max|dSFR| = 0
benchmark_mean_filter:  4.6-48.5x,  max|d|   = 0
benchmark_conv_matrix:
  构建  2.4-22.3x,  max|dK| = 0
  批量  1.4-4.3x
benchmark_likelihood_interp:
  插值  1.3-2.8x,  max|d| = 0
  call_batch 1.5-2.1x,  rel ≈ 6e-16
benchmark_csp:
  合成  1.9-2.3x
  真实  1.5-16.5x (N=400: 4.13 s -> 0.251 s),  rel ≈ 3e-15
benchmark_specfit_hotpath:
  calz 2.6x | model1 1.54x | mode2 10.30x | both 1.74x | 真实谱 4.11x
```

6. 合并前 vs 合并后（重点）

指标	合并前 332a043	合并后 6154fd6	说明
JAX 加速比	2.3-3.3×	3.1-4.0×	master 的 1D 卷积核
JAX N=1000 稳态	0.286 s	0.181 s	内存/计算都更省
主 MCMC 墙钟	88 s	74 s	更快
DPL 墙钟	437 s	511 s	似然调用 59.3万→131万（采样路径变化，非变慢）
端到端总墙钟	547 s	607 s	主要被 DPL 的调用量拉长
峰值内存	3.16 GiB	3.92 GiB	见下
Pair/数值一致性	~1e-15	~1e-15	新旧实现仍为浮点级一致

说明： - 端到端总时间变长**不代表代码变慢**。JAX 单次评估明显更快（benchmark_jax 与主 MCMC 都更快）；DPL 段只是这次 UltraNest 走了不同的加宽路径、多评估了一倍多的似然，带随机性。 - 峰值内存升到 3.92 GiB，与 master 引入的 likelihood_jax / notebook 加载 / 运行期缓存有关，仍在机内存范围内（无 swap）。 - 三次 UltraNest 现在**各自独立写 debug.log**（out/chains_*/debug.log 三份都在），验证了此前的日志修复在真实多 run 场景生效。

7. 生成的实际产物

- **图片：** out/figs/01...07b （10 张 PNG，均由本次运行重新生成）
- **数据：** out/data/specfit_result.fits、out/data/mcmc_bestfit.fits、out/data/posterior_samples.npy
- **链：** out/chains_manga-7443/、out/chains_custom_sf/、out/chains_dpl/ （各含独立 debug.log）
- **日志：** out/example_run.log、out/benchmarks/*.log （含 ALL.log、pytest.log）

8. 发现与建议

1. **DPL 段仍是唯一瓶颈**（约 84% 墙钟、131 万次似然）。不需模型对比时删掉本节，全流程可压到约 1.5 分钟。

2. **JAX 1D 卷积核**是本次合并的性能亮点：加速比 $2.3\text{--}3.3\times \rightarrow 3.1\text{--}4.0\times$ ，单次评估更快。
3. **JAX 与 NumPy 仍差约 $5e-4$** （JAX 默认 float32）——MCMC 采样走 float32，最终绘图走 NumPy float64。
4. **DPL 的 beta 仍约束很弱**（corner 报 "Too few points to create valid contours"；本次还出现 "Sampling from region seems inefficient" 警告），属该模型/数据的固有困难。
5. **CPU 利用率 307%**：嵌套采样本身偏串行，未吃满 8 线程；调线程数收益有限。
6. **benchmark_conv_matrix 的批量项仍只有 $1.4\text{--}4.3\times$** ：本机 K₈₀ 大矩阵乘占主导，与 80 核基线的 9–92 $\times$ 差异属硬件相关。

9. 复现命令

```
source ~/miniconda3/etc/profile.d/conda.sh
conda activate bigspy
cd /home/chengz/Code/bigspy

# 单元测试
python -m pytest -q

# 端到端全流程
MPLBACKEND=Agg /usr/bin/time -v python example/run_bigspy_jax.py

# 全部 benchmark（串行，约 5 分钟）
for b in benchmark_sfh benchmark_csp benchmark_conv_matrix \
    benchmark_likelihood_interp benchmark_mean_filter \
    benchmark_specfit_hotpath benchmark_mcmc_pipeline benchmark_jax; do
    echo "== $b =="; python "benchmarks/$b.py"
done
```